

KONDA

Детальный разбор учебного проекта

Дата: 4 июня 2026

Проект: Konda

Репозиторий: <https://github.com/daniliammo/Konda>

1. Кратко о проекте

Konda - это учебный экспериментальный язык программирования с русским синтаксисом. Сейчас он работает как транpiler: программа на языке Konda переводится в обычный C-код, после чего этот C-код компилируется системным C-компилятором в запускаемый файл.

Главная идея проекта - показать, как устроен путь от текста программы до исполняемого файла:

файл .конда

- > чтение исходного текста
- > токенизация
- > разбор простого синтаксиса
- > генерация C-кода
- > компиляция C
- > готовая программа

Для первого курса колледжа это сильная и живая идея: проект показывает не только владение C, но и понимание основ языков программирования, компиляторов, ошибок, типов и генерации кода.

2. Для чего нужен проект

Проект не стоит подавать как замену C, Python или Rust. Сильнее и честнее позиционировать его как учебный язык и транpiler.

Цели проекта:

1. Понять, как компьютер разбирает текст программы.
2. Научиться строить лексер: модуль, который превращает текст в токены.
3. Научиться строить простой парсер: модуль, который понимает конструкции языка.
4. Научиться генерировать код другого языка, в данном случае C.
5. Научиться показывать понятные ошибки пользователю.
6. Постепенно прийти к синтаксическому дереву AST и семантическому анализу.

Практическая ценность в том, что проект можно показывать: есть исходный файл на Konda, есть сгенерированный C-файл, есть собранная программа, есть тесты.

3. Пример программы

Пример исходника на Konda:

```
#содержит <stdio.h>
```

```
целое4 точка_входа(целое4 количество_аргументов, символ** аргументы)
{
    если (количество_аргументов > 1) {
        printf(аргументы[1]);
    } иначе {
        printf("аргументов нет");
    }
}
```

```
}  
}
```

Примерно такой C-код генерируется:

```
#include <stdint.h>  
#include <stdio.h>  
  
int32_t main(int32_t argc, char **argv)  
{  
    if (argc > 1) {  
        printf("%s\n", argv[1]);  
    } else {  
        printf("%s\n", "аргументов нет");  
    }  
    return 0;  
}
```

Важная мысль: Konda пока не исполняется сама по себе. Она переводится в C, а уже C компилируется и запускается.

4. Что уже есть в проекте

В проекте уже есть рабочая основа:

- чтение файла .конда;
- лексер, который разбивает текст на токены;
- набор типов токенов;
- динамический массив токенов;
- простой транpiler Konda -> C;
- специальная обработка точки входа;
- поддержка русских ключевых слов;
- генерация C-файла;
- запуск C-компилятора;
- тестовый пример;
- автотесты;
- README с описанием сборки и текущих возможностей.

Это уже больше, чем просто идея. Это минимально работающий прототип языка.

5. Основные файлы проекта

README.md

Объясняет, что такое Konda, как собрать проект, как запустить пример, какие возможности уже есть и что планируется дальше.

Makefile

Описывает сборку транпилятора. После правок сборка стала переносимее: используется cc по умолчанию, а Linux-специфичные флаги применяются только на Linux.

транспилятор.h

Общий заголовочный файл. В нем описаны типы токенов, структура токена, структура массива токенов и публичные функции проекта.

токенизатор_лексер.c

Лексер. Он проходит по исходному тексту и выделяет токены: идентификаторы, числа, строки, операторы, скобки, комментарии, ключевые слова.

массив_токенов.с

Динамический массив токенов. Нужен, потому что заранее неизвестно, сколько токенов получится из исходного файла.

ввод_вывод.с

Читает исходный файл в память. Сейчас добавлены проверки ошибок: если файл не найден или не читается, программа не должна падать.

транспилиция.с

Главный модуль текущей генерации С. Он идет по токенам, понимает простые конструкции языка и сразу печатает С-код в буфер.

основа.с

Точка входа самого транпилятора. Принимает путь к .конда файлу, читает его, токенизирует, транпилирует и запускает компиляцию С.

tests/run.sh

Минимальные автотесты. Они проверяют, что проект собирается, пример запускается, условие $\geq$ работает, небезопасный доступ к аргументам ловится, а несуществующий файл обрабатывается ошибкой.

6. Как работает конвейер

Шаг 1. Пользователь запускает транпилятор:

```
./Собранное/Транспилятор тест.конда
```

Шаг 2. Модуль чтения файла загружает текст программы в память.

Шаг 3. Лексер превращает текст в массив токенов.

Например:

```
если (количество_аргументов > 1)
```

превращается примерно в:

```
ТОКЕН_ЕСЛИ  
ТОКЕН_КРУГЛАЯ_СКОБКА_ОТКРЫВАЕТСЯ  
ТОКЕН_ИДЕНТИФИКАТОР  
ТОКЕН_СТРЕЛКА_ВПРАВО  
ТОКЕН_ЧИСЛО  
ТОКЕН_КРУГЛАЯ_СКОБКА_ЗАКРЫВАЕТСЯ
```

Шаг 4. Транспилятор проходит по токенам и генерирует С.

Например:

```
если (...) { ... } иначе { ... }
```

становится:

```
if (...) { ... } else { ... }
```

Шаг 5. Получившийся С-файл компилируется командой `cc`.

Шаг 6. Пользователь получает исполняемый файл с расширением `.elf`.

7. Важная особенность: проверка границ аргументов

В проекте уже есть интересная идея: проверять доступ к аргументам программы на этапе трансляции.

Например, этот код считается безопасным:

```
если (количество_аргументов > 1) {  
    printf(аргументы[1]);  
}
```

Почему? Потому что если количество_аргументов больше 1, значит есть аргумент с индексом 1.

А такой код должен быть ошибкой:

```
если (количество_аргументов > 0) {  
    printf(аргументы[1]);  
}
```

Потому что условие `> 0` гарантирует только `argv[0]`, но не `argv[1]`.

Это хорошая учебная демонстрация статической проверки: программа может поймать часть ошибок до запуска.

8. Что такое AST и почему сын про него говорит

AST - это *abstract syntax tree*, или абстрактное синтаксическое дерево.

Сейчас транслятор в основном "пробегаются" по токенам и сразу генерирует С. Для маленького набора конструкций это нормально. Но когда язык растет, такой подход становится неудобным.

AST нужен, чтобы программа сначала построила дерево смысла:

```
функция  
  блок  
    если  
      условие  
      then-блок  
      else-блок
```

А уже потом отдельный модуль обходит это дерево и генерирует С.

Плюсы AST:

- легче делать понятные ошибки;
- легче проверять типы;
- легче поддерживать приоритеты операторов;
- легче добавлять `return`, циклы, функции, массивы;
- легче делать анализ владения памятью;
- легче тестировать парсер отдельно от генерации С.

Мнение: AST действительно нужен, но не как самый первый шаг. Сначала правильно стабилизировать сборку, тесты и диагностику. Потом строить AST, чтобы не

сломать уже работающее поведение.

9. Что уже было улучшено

В текущей рабочей версии сделаны первые инфраструктурные улучшения:

1. Makefile стал переносимее между macOS и Linux.
2. Транспилятор больше не использует жесткий путь `/bin/cc`.
3. Чтение файла больше не падает на несуществующем файле.
4. Ошибки парсера стали информативнее: видно, что ожидалось и что найдено.
5. Условие `>=` теперь обрабатывается корректнее.
6. Добавлены автотесты.
7. README стал полноценным описанием проекта.

Команда проверки:

```
make test
```

Ожидаемый результат:

```
OK: все тесты прошли
```

10. Сильные стороны проекта

1. Идея необычнее стандартных учебных проектов.

Для первого курса это выглядит сильнее, чем простой калькулятор, сайт или бот, потому что затрагивает устройство языков программирования.

2. Есть рабочий результат.

Проект не только описан словами. Он реально читает `.konda` файл, генерирует C и собирает программу.

3. Хорошая образовательная ценность.

Тут есть C, работа со строками, памятью, файлами, массивами, структурами, диагностикой ошибок и сборкой.

4. Есть понятная траектория развития.

Можно развивать проект постепенно: тесты, диагностика, AST, типы, циклы, функции, массивы, владение памятью.

5. Русский синтаксис делает проект заметным.

Это помогает объяснять проект людям, которые не погружены в компиляторы.

11. Слабые места и риски

1. Нельзя обещать слишком много.

Konda пока не промышленный язык и не замена существующим языкам. Лучше честно говорить: это учебный транспилятор и эксперимент.

2. Без AST проект быстро станет трудно расширять.

Прямой проход по токенам подходит для старта, но дальше потребуется синтаксическое дерево.

3. Нужны тесты на каждую новую возможность.

Иначе любое изменение в парсере может незаметно сломать старые примеры.

4. Нужно убрать из репозитория собранные бинарники.

Исходники должны храниться в git, а .o, .elf и собранный транслятор лучше генерировать локально.

5. Нужно аккуратно формулировать цель.

Самая сильная формулировка: "учебный расширяемый русский язык, который транпилируется в C".

12. Что можно показать преподавателю или знакомым

Короткая формулировка:

Konda - учебный русский язык программирования. Его код переводится в C, а затем компилируется в обычную программу. Проект показывает, как устроены лексер, парсер, генерация кода и диагностика ошибок.

Что показать на демонстрации:

1. Открыть файл тест.конда.
2. Показать русские ключевые слова.
3. Запустить транслятор.
4. Открыть сгенерированный тест.с.
5. Запустить собранный тест.elf.
6. Показать ошибку на неправильном доступе к аргументу.
7. Запустить make test.

Это будет понятная демонстрация "от исходника до работающей программы".

13. Рекомендуемый план развития

Этап 1. Чистка проекта

- убрать бинарники из git;
- убрать .idea из git;
- оставить понятную структуру исходников;
- сохранить README и tests.

Этап 2. Расширение тестов

- тест лексера;
- тест успешной трансляции;
- тест ошибок синтаксиса;
- тест ошибок границ;
- тест разных примеров языка.

Этап 3. Улучшение диагностики

- строка и колонка;
- найденный токен;
- ожидаемый токен;
- фрагмент исходной строки;
- подсказка, что исправить.

Этап 4. AST

- отдельные структуры для программы, функции, блока, оператора, выражения;
- парсер строит AST;
- генератор C обходит AST;
- тесты проверяют дерево или результат генерации.

Этап 5. Новые возможности языка

- return;
- циклы;
- присваивания;
- функции;
- несколько типов параметров;
- массивы;
- базовая таблица символов;
- проверка типов.

Этап 6. Более амбициозные идеи

- владение памятью;
- autofree;
- статические проверки безопасности;
- генерация ASCII-имен для C;
- документация языка.

14. Итоговое мнение

Проект имеет хорошую задумку и вполне подходит для первого курса колледжа. Главное - держать объем реалистичным.

Правильная подача:

"Я делаю учебный русский язык программирования Konda. Он транспируется в C. На проекте я показываю, как работают лексер, парсер, генерация кода, компиляция и диагностика ошибок."

Это звучит сильно, честно и профессионально. Проект можно довести до хорошей защиты, если не пытаться сразу сделать большой промышленный язык, а постепенно развивать маленькое, но рабочее ядро.